

Multiple inheritance (in C++ and Java)

1

```
#include <iostream>
using namespace std;

class Vehicle {
public:
    Vehicle() {cout << "Vehicle Constructor" << endl;}
    virtual ~Vehicle() {cout << "Vehicle Destructor" << endl;}

    virtual void accelerate() const {cout << "Vehicle Accelerating" << endl;}

    void setAcceleration(double a) {acceleration = a;}
    double getAcceleration() const {return acceleration;}

private:
    double acceleration;
};

class Car: public Vehicle {
public:
    Car() {cout << "Car Constructor" << endl;}
    virtual ~Car() {cout << "Car Destructor" << endl;}

    virtual void accelerate() const {cout << "Car Accelerating" << endl;}
    void drive() const {cout << "Car Driving" << endl;}

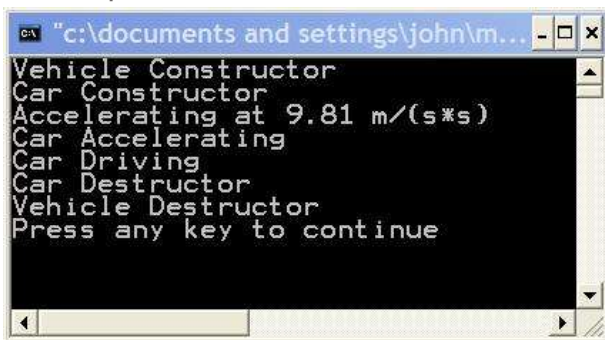
private:
    // Car inherits acceleration accessors, member
};

int main() {
    Car myCar;
    myCar.setAcceleration(9.81);
    //One "G"
    cout << "Accelerating at " <<
        myCar.getAcceleration() << " m/(s*s)";
    cout << endl;

    myCar.accelerate();
    myCar.drive();
}
```

2

Output



```
Vehicle Constructor
Car Constructor
Accelerating at 9.81 m/(s*s)
Car Accelerating
Car Driving
Car Destructor
Vehicle Destructor
Press any key to continue
```

3

Note

- Accelerate method is declared as virtual. Any method that will be overridden in a subclass should be declared as virtual.
- Notice that in the Car class I have used the keyword virtual as a specifier on the Accelerate method. This is optional. **Once a method is declared as virtual, it is automatically virtual in all subclasses. But, by using the keyword "virtual" in Car, it is clearer that Accelerate() is a virtual method.** Another programmer, looking only at the Car class, would know that the method was virtual.
- setAcceleration and getAcceleration, are non virtual. Why? I don't expect to override them in any subclasses, but rather just inherit them. Notice that the Car class has indeed inherited these methods.

4

```
class JetCar: public Car {
public:
    JetCar() {}
    virtual ~JetCar() {}

    virtual void drive() const {cout <<
        "JetCar driving" << endl;}
    virtual void fly() const {cout << "JetCar
        flying" << endl;}
};

-----
some function elsewhere
-----

void analyzePerformance(Car
    *testVehicle)
{
    testVehicle->drive();
    testVehicle->fly();
}
```

5

- the drive() method of JetCar override the the drive() of car because the driving method of jetcar should be a little different from car
- because JetCar can fly, so a new method called fly() is added and is prepared to be overridden

- There is an error in compilation in analyzePerformance
- > car does not have a fly method only JetCar does

Percolating Up

```
class Car: public Vehicle {
public:
    Car() {}
    virtual ~Car() {}

    virtual void accelerate() const {cout << "Car Accelerating" << endl;}
    virtual void drive() const {cout << "Car Driving" << endl;}
    virtual void fly() const {cout << "Cars can only fall" << endl;}

private:
    ??? Car inherits acceleration accessors, member
};
```

6

```
class JetCar: public Car {
public:
    JetCar() {}
    virtual ~JetCar() {}

    virtual void drive() const {cout << "JetCar driving" << endl;}
    virtual void fly() const {cout << "JetCar flying" << endl;}
};

int main() {
    Car myCar;
    JetCar myJetCar;
    analyzePerformance(&myCar);
    analyzePerformance(&myJetCar);
    return 0;
}
```

Percolating upward

- We get the correct result.
- The Car class has a "fly" method. Surely, Cars don't fly. We added the fly method to Car so that a base class pointer could be used on either. This is known as percolating upward.
- The fly method was moved up into a base class, but the base class Car should not have that functionality.
- If we percolated up more methods as we built these classes, Car becomes bloated and confused. Generally, only functionality common to all subclasses belongs in a base class.

```

c:\documents and settin...
Car Driving
Cars can only fall
JetCar driving
JetCar flying
Press any key to continue
  
```

7

A smarter solution

```

class Car: public Vehicle {
public:
    Car() {}
    virtual ~Car() {}
    virtual void accelerate() const {cout << "Car Accelerating" << endl;}
    virtual void drive() const {cout << "Car Driving" << endl;}
private:
    // Car inherits acceleration
    // accessors, member
};

class JetCar: public Car {
public:
    JetCar() {}
    virtual ~JetCar() {}

    virtual void drive() const {cout << "JetCar driving" << endl;}
    virtual void fly() const {cout << "JetCar flying" << endl;}
};

void analyzePerformance(Car *testVehicle)
{
    JetCar *ptr;
    //A pointer of subclass type, JetCar
    testVehicle->drive();
    //drive() exists for both base and sub class
    ptr = dynamic_cast<JetCar*>(testVehicle);
    if (ptr) { ptr->fly(); }
    else { cout << "Car being tested" << endl; }
}

int main() {
    Car myCar;
    JetCar myJetCar;

    analyzePerformance(&myCar);
    analyzePerformance(&myJetCar);
    return 0;
}
  
```

8

A good solution?

- C++ has a built in mechanism for **Runtime Type Identification, RTTI**. This means that as a program is running it is possible to determine the exact type of object a pointer or reference refers to.
- RTTI is achieved in C++ via the **dynamic_cast** operator, the typeid operator and the type_info class.
- The **dynamic_cast** operator returns a pointer to type JetCar if have the address of a JetCar object in the original pointer, testVehicle. Otherwise, the dynamic_cast operator returns NULL.
- Dynamic casting used in this manner is referred to as "**casting down**". We have casted a base class pointer down its hierarchy into a subclass pointer.

9

How about Java

- In Java you can use **instanceof**

- For example

```

analyzePerformance (Car testvehicle) {
    if (testvehicle instanceof JetCar) {
        .....
    }
}
  
```

10

bad design

- Explicitly checking an objects type is usually indicative of bad design
- Generally, you should avoid explicitly testing the type of an object in your code.
- Why? As told in **spaghetti code course**. As additional classes are formed, every point in the code that makes explicit checks on object type may need modification. If you forget to modify code somewhere, you may not find out until your program fails and your JetCar careens off into the sun. For practice, consider what modifications would be needed to support my next project, the BoatCar.

11

Casting in C and C++

```

ptr = dynamic_cast <JetCar *>(testVehicle);
//Dynamic Cast

ptr = static_cast <JetCar *>(testVehicle); //Static Cast

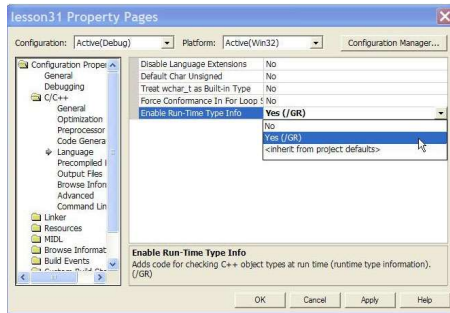
ptr = (JetCar *) testVehicle; //C style static cast
  
```

Important Note: In Visual C++, RTTI is not turned on by default, if you run this code without turning on RTTI, you will get a cryptic error message saying something like "dynamic_cast' used on polymorphic type 'Car' with /GR-; unpredictable behavior may result". Turning on RTTI allows the compiler to insert additional code to support RTTI as it forms object files.

12

Turn on RTTI in Visual Studio.net

- go to click solution -> project -> properties



13

The problem revisited

- A JetCar is really both a Jet and a Car. We have tried inheriting from Car alone, but this isn't really satisfactory.
- Our JetCar may need many properties (members) and abilities (methods) of a Jet. We need to inherit those as well. This can be done with multiple inheritance. Here is the syntax.

14

the right solution

```
#include <iostream>
using namespace std;
class Vehicle {
public:
    Vehicle() {cout << "Vehicle
    Constructor" << endl;}
    virtual ~Vehicle() {cout <<
    "Vehicle Destructor" << endl;}

    virtual void accelerate() const
    {cout << "Vehicle Accelerating" <<
    endl;}
    void setAcceleration(double a)
    {acceleration = a;}
    double getAcceleration() const
    {return acceleration;}

protected:
    double acceleration;
};
```

```
class Car: public Vehicle {
public:
    Car() {cout << "Car Constructor" << endl;}
    virtual ~Car() {cout << "Car Destructor" <<
    endl;}

    virtual void accelerate() const {cout << "Car
    Accelerating" << endl;}
    virtual void drive() const {cout << "Car Driving"
    << endl;}

private:
    // Car inherits acceleration accessors, member
};

class Jet: public Vehicle {
public:
    Jet() {cout << "Jet Constructor" << endl;}
    virtual ~Jet() {cout << "Jet Destructor" <<
    endl;}

    virtual void fly() const {cout << "Jet flying" <<
    endl;}

};

class JetCar: public Car, public Jet {
public:
    JetCar() {cout << "JetCar Constructor" <<
    endl;}
    virtual ~JetCar() {cout << "JetCar Destructor"
    << endl;}
};
```

15

```
class JetCar: public Car, public Jet {
public:
    JetCar() {cout << "JetCar Constructor"
    << endl;}
    virtual ~JetCar() {cout << "JetCar
    Destructor" << endl;}

    virtual void drive() const {cout <<
    "JetCar driving" << endl;}
    virtual void fly() const {cout << "JetCar
    flying" << endl;}
};

void analyzeCarPerformance(Car
*testVehicle)
{
    testVehicle->drive();
    //drive() exists for both base and sub
    class
}

void analyzeJetPerformance(Jet
*testVehicle)
{
    testVehicle->fly();
    //fly() exists for both base and sub class
}

int main() {
    Car myCar;
    Jet myJet;
    JetCar myJetCar;

    cout << endl << endl;
    cout << "Car testing in progress" << endl;
    analyzeCarPerformance(&myCar);
    analyzeCarPerformance(&myJetCar);

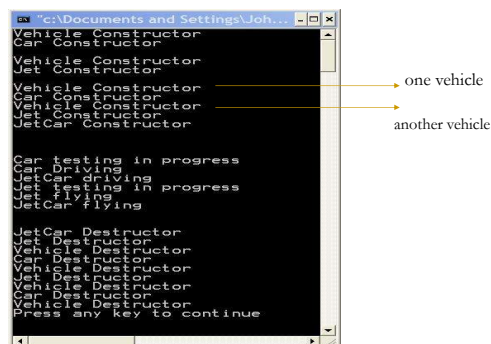
    cout << "Jet testing in progress" << endl;
    analyzeJetPerformance(&myJet);
    analyzeJetPerformance(&myJetCar);
    cout << endl << endl;

    return 0;
}
```

16

15

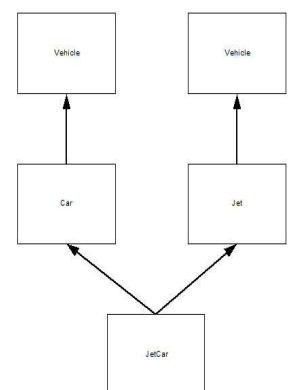
16



17

17

- JetCar inherits from both Car and Jet.
- JetCar inherits two Vehicle parts. This means that when calling a method that is in Vehicle, but not overridden in the subclass JetCar, or when we access a variable that exists only in Vehicle, we must explicit specify which Vehicle part we are accessing: **we must explicit specify which Vehicle part we are accessing:**



18

18

Summary

- with multiple inheritance a subclass is a kind of each of its base classes. **JetCar** is a **Car** and it is a **Jet**. It may be used in places that expect a Car object, such as **analyzeCarPerformance**. It can also be used in places that expect a Jet, such as **analyzeJetPerformance**.
- However, how about the ambiguity?

19

19

歧異辯析

- Ambiguity can happen when you use multiple inheritance
- 有許多方法可以解決 ambiguity

20

20

Solution 1 – casting ambiguous attributes or methods

```
class task {
    // .....
    virtual debug_info* get_debug();
};
class Displayed {
    // .....
    virtual debug_info* get_debug();
};
class Satellite : public task, public
displayed {
    // .....
};

void print_debugging(Satellite *sp) {
    debug_info * dip =
        sp-> get_debug(); //wrong
    debug_info *dip =
        sp->Task::get_debug();
    debug_info *dip =
        sp->Displayed::get_debug();
}
```

21

21

Solution 2 – 覆寫掉

```
class Satellite: public Task, public Displayed {
    //...

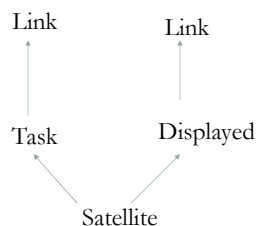
    debugInfo* get_debug () // override Task Displayed get_debug {} {
        debug_info* dip1 = Task::get_debug();
        debug_info* dip2 = Displayed::get_debug ();
        return dip->merge(dip2); //write a new function to merge
        // the debugging messages from two sources
    };
};
```

22

22

Another case like JetCar

```
class Link {
    Link * next ;
};
class Task: public Link {
    // using link to maintain a
    list of tasks
};
class Displayed: public Link {
    // use link to maintain a
    list of displayed object
};
```



23

23

解決重複基底類別

```
void mess_with_links(Satellite *p) {
    p-> next = 0 ; // wrong
    p-> Link::next ; // wrong
    p-> Task::Link::next = 0 ; // OK
    p-> Displayed:: Link:: next = 0 ; // OK
}
```

24

24

Another example

```
class Storable {
public:
    virtual const char*
        get_file () =0;
    virtual void read () =0;
    virtual void write ()
        =0;
    virtual ~Storable () {}
};

class Transmitter : public Storable {
public:
    void write () ;
    // ...
};

class Receiver : public Storable {
public: void write () ;
    // ...
};

class Radio : public Transmitter,
              public Receiver {
public:
    const char* get_file () ;
    void read () ;
    void write () ;
    // ...
};
```

想像 storable 是一個能自檔案讀出自己
將自己寫回檔案的物件

25

25

```
void Radio::write () {
    Transmitter::write () ;
    Receiver::write () ;
    // begin to write something that is really related
    // to radio
}
```

26

26

```
class Storable {
public:
    Storable (const char* s) ;
    // 每一個 storable 用一個 file
    // 來記錄
    virtual void read () =0;
    virtual void write () =0;
    virtual ~Storable () ;
private:
    const char* store;
    Storable (const Storable & ) ;
    Storable& operator= (const Storable
    );
};

class Radio : public Transmitter, public Receiver
{
public:
    // ...
};
```

有這樣的改變，下面的 radio
繼承就會出問題，會有兩份基底
但是一個 radio 不可能存到兩份
file

27

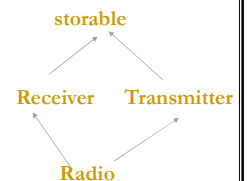
27

Solution

```
class transmitter: public virtual Storable {
public:
    void write();
};

class Receiver:: public virtual Storable {
public:
    void write();
};

class Radio: public Transmitter, public Receiver {
public:
    void write();
};
```



鑽石形繼承
diamond-shaped inheritance

28

28

Java Interface

- Java think ambiguity problem caused by multiple inheritance is too complicated.
- Most C++ programmers have problems to understand
- Java prohibits multiple inheritance
- BUT, it introduce **interface**

29

29

The example

Suppose you already write a class

```
Class myLinkedList extend LinkedList {

    void appendTail(node o) ;
    void appendHead(node o);
    void appendMiddle(node left, node right);

}
```

30

30

How do you make your myLinkedList be used as Stack, Queue

```
Interface Stack {
    item pop();
    void push(o);
};
```

```
Interface Queue {
    enqueue(o);
    item dequeue();
}
```

31

31

Class myLinkedList extend LinkedList **implement** Stack, Queue

```
void appendTail(node o) ;
void appendHead(node o);
void appendMiddle(node left, node right);
pop() { .... }
push() { .....}
enqueue() { ....}
dequeue() { ..... }
```

32

32

How to use Java interface?

- Somewhere in initialization code
Stack s = new myLinkedList();
- Somewhere in core
s.push(o);
.....
o = s.pop();

33

33

Discussion

- To understand Java interface, it is better understood from multiple inheritance. **Implement** is a concept of purified multiple inheritance
- **However, how about the inheritance Principle**
 - **Class A: B, C**
 - A is B
 - A is C
 - **Class A extend B implement C, D**
 - A is B
 - A is C?
 - A is D?

34

34

Discussion

- In principle, implementing an interface has loosen inheritance principle to follow because there is no attributes (variables) to inherit from **implement**
- **However, think of the follow usage**
 - Stack s = new MyLinkedList();
 - MyLinkedList is a Stack?

35

35

To simulate interface in C++

- Class human {
virtual void walk() = 0 ;
Virtual void breath() = 0;
}

36

36